Typing Rules, Domain Semantics, and Pi Injectivity for Dependent Type Theory with $\mathsf{U} : \mathsf{U}$

1 Judgments

We work with three mutually defined judgment forms:

$\Gamma\ \mathsf{ctx}$	Γ is a well-formed context
$\Gamma \vdash M : A$	M has type A in context Γ
$\Gamma \vdash M = N : A$	M and N are definitionally equal at type A

Contexts are snoc-lists: $\Gamma ::= \cdot \mid \Gamma, x{:}A$. We write $M\sigma$ for the result of applying a substitution σ to a term M, and $[t]$ for the substitution $\Gamma \to \Gamma, x{:}A$ extending the identity by t, so that $B[a]$ means $B(\mathrm{id}, a)$.

2 Well-formed contexts

$$\frac{}{\cdot\ \mathsf{ctx}} \qquad\qquad \frac{\Gamma \vdash A : \mathsf{U}}{\Gamma, x{:}A\ \mathsf{ctx}}$$

3 Typing rules

$$\textsc{Var}\ \frac{\Gamma\ \mathsf{ctx} \qquad (x{:}A) \in \Gamma}{\Gamma \vdash x : A} \qquad \textsc{Conv}\ \frac{\Gamma \vdash M : A \qquad \Gamma \vdash A = B : \mathsf{U} \qquad \Gamma \vdash B : \mathsf{U}}{\Gamma \vdash M : B} \qquad \mathrm{U}\ \frac{\Gamma\ \mathsf{ctx}}{\Gamma \vdash \mathsf{U} : \mathsf{U}}$$

$$\textsc{Pi}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U}}{\Gamma \vdash \Pi(x{:}A)B : \mathsf{U}} \qquad \textsc{Lam}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma, x{:}A \vdash M : B}{\Gamma \vdash \lambda(x{:}A)M : \Pi(x{:}A)B}$$

$$\textsc{App}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash f : \Pi(x{:}A)B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B[a]}$$

4 Conversion rules

$$\textsc{Refl}\ \frac{\Gamma \vdash M : A}{\Gamma \vdash M = M : A} \qquad \textsc{Sym}\ \frac{\Gamma \vdash M = N : A}{\Gamma \vdash N = M : A} \qquad \textsc{Trans}\ \frac{\Gamma \vdash M = N : A \qquad \Gamma \vdash N = P : A}{\Gamma \vdash M = P : A}$$

$$\textsc{Conv-Eq}\ \frac{\Gamma \vdash M = N : A \qquad \Gamma \vdash A = B : \mathsf{U} \qquad \Gamma \vdash B : \mathsf{U}}{\Gamma \vdash M = N : B}$$

$$\textsc{Beta}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma, x{:}A \vdash M : B \qquad \Gamma \vdash a : A}{\Gamma \vdash (\lambda(x{:}A)M)\ a = M[a] : B[a]}$$

$$\textsc{Pi-Cong}\ \frac{\Gamma \vdash A = A' : \mathsf{U} \qquad \Gamma, x{:}A \vdash B = B' : \mathsf{U}}{\Gamma \vdash \Pi(x{:}A)B = \Pi(x{:}A')B' : \mathsf{U}}$$

Note that in Pi-Cong the codomain premise compares B and B' in the context extended by the *left* domain A, not by A'. This is the standard convention: since $A = A' : \mathsf{U}$ holds, one could equivalently use A' (via the conversion rule on contexts), but fixing the left side avoids a mutual dependency between the congruence rule and context conversion.

We write M^+ for the weakening of a term M by one fresh variable, i.e. M^+ is M viewed in the extended context $\Gamma, x{:}A$ where it does not depend on x.

$$\textsc{Funext}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash f^+\ x = g^+\ x : B \qquad \Gamma \vdash f : \Pi(x{:}A)B \qquad \Gamma \vdash g : \Pi(x{:}A)B}{\Gamma \vdash f = g : \Pi(x{:}A)B}$$

$$\textsc{App-Fun}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash f = f' : \Pi(x{:}A)B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a = f'\ a : B[a]}$$

$$\textsc{App-Arg}\ \frac{\Gamma \vdash A : \mathsf{U} \qquad \Gamma, x{:}A \vdash B : \mathsf{U} \qquad \Gamma \vdash f : \Pi(x{:}A)B \qquad \Gamma \vdash a = a' : A}{\Gamma \vdash f\ a = f\ a' : B[a]}$$

Theorem 1 (Eta-conversion)**.** *If* $\Gamma \vdash A : \mathsf{U}$, $\Gamma, x{:}A \vdash B : \mathsf{U}$, *and* $\Gamma \vdash f : \Pi(x{:}A)B$, *then*

$$\Gamma \vdash f = \lambda(x{:}A)f^+\ x : \Pi(x{:}A)B.$$

Proof. By Funext it suffices to show $\Gamma, x{:}A \vdash f^+\ x = (\lambda(x{:}A)f^+\ x)^+\ x : B$. Since weakening distributes under λ, we have $(\lambda(x{:}A)M)^+ = \lambda(x{:}A^+)M^+$ (where the inner weakening acts on the next-to-last variable, skipping the binder). The right-hand side is therefore a β-redex. Applying Beta and the substitution identity $M^+[x_0] = M$ gives the result. Formalised in `EtaConversion.agda`. □

5 Head reduction

We define head reduction $M \rightsquigarrow N$ as the compatible closure of β-reduction at the head:

$$\textsc{HR-Beta}\ \frac{}{(\lambda(x{:}A)M)\ a \rightsquigarrow M[a]} \qquad \textsc{HR-App}\ \frac{M \rightsquigarrow M'}{M\ a \rightsquigarrow M'\ a}$$

We write $M \rightsquigarrow^* N$ for the reflexive-transitive closure. The validity relation uses a wrapper $\mathsf{Red}\ \Gamma\ M\ N\ A$, which carries only a proof of $M \rightsquigarrow^* N$; the context Γ and type A are phantom parameters kept for inference convenience. In [1], the reduction relation is defined as a subrelation of conversion, requiring both $M \rightsquigarrow^* N$ and $\Gamma \vdash M = N : A$. Here, reduction is purely syntactic: the conversion witnesses are instead stored directly at the leaves of the bundled validity relation (Val2/EqVal2), so the Red wrapper need not carry them.

6 Finite elements and the information ordering

Following Coquand–Huber [1], we interpret types and terms as ideal-domain elements, approximated from below by *finite elements*. The set FinEl and its auxiliary FinFun are defined by mutual induction:

$$\begin{aligned} u, v, a, b &::= \bot \mid \mathsf{U} \mid \mathsf{Fun}(g) \mid \mathsf{Pi}(a, f) \\ g, f &\in \mathsf{FinFun} = \mathsf{List}(\mathsf{FinEl} \times \mathsf{FinEl}) \end{aligned}$$

Here $\bot$ is the least element (no information), U is the code for the universe, $\mathsf{Fun}(g)$ is a function element with graph g (a finite list of input–output pairs), and $\mathsf{Pi}(a, f)$ is a code for a Π-type with domain approximation a and codomain graph f.

Binary supremum. A partial binary supremum $u \sqcup v$ is defined by case analysis: it combines compatible codes ($\mathsf{Pi}(a, f) \sqcup \mathsf{Pi}(b, g) = \mathsf{Pi}(a \sqcup b, f +\!\!+ g)$, $\mathsf{Fun}(g) \sqcup \mathsf{Fun}(h) = \mathsf{Fun}(g +\!\!+ h)$) and returns $\bot$ for incompatible pairs (e.g. $\mathsf{U} \sqcup \mathsf{Fun}(g) = \bot$).

Evaluation of finite functions. Given a finite function $f = [(u_1, v_1), \ldots, (u_k, v_k)]$ and an input u, the sup-based evaluation collects all outputs whose keys are below u:

$$\mathsf{ev}(f, u) = \bigsqcup \{v_i \mid u_i \leq u\}.$$

Information ordering. The ordering $u \leq v$ on FinEl is defined mutually with an ordering on finite functions. The key clauses are:

$$\begin{aligned} \bot \leq v &= \top \\ \mathsf{U} \leq \mathsf{U} &= \top \\ \mathsf{Fun}(g) \leq \mathsf{Fun}(h) &\iff g \leq_{\mathsf{fun}} h \\ \mathsf{Pi}(a, f) \leq \mathsf{Pi}(b, g) &\iff a \leq b \ \wedge\ f \leq_{\mathsf{fun}} g \end{aligned}$$

where the ordering on finite functions uses the *finite condition*:

$$g \leq_{\mathsf{fun}} h \iff \forall (u_i, v_i) \in g.\ v_i \leq \mathsf{ev}(h, u_i).$$

All other combinations (e.g. $\mathsf{U} \leq \mathsf{Fun}(g)$) are empty.

Coherence. A finite element is *coherent* ($\mathsf{Coh}(u)$) when its internal data is consistent. Specifically, $\mathsf{Coh}(\mathsf{Pi}(a, f))$ requires $\mathsf{Coh}(a)$ and that f is a *coherent finite function*: whenever two edges $(u_1, v_1), (u_2, v_2) \in f$ have compatible keys ($\mathsf{Comp}(u_1, u_2)$), their values are also compatible ($\mathsf{Comp}(v_1, v_2)$). This condition ensures that finite functions represent well-defined continuous maps between domains.

Finite membership. The *finite typing* relation $u : a$ ("u is an element of the type coded by a") is defined by mutual recursion on u and a:

$$\begin{aligned} \bot : a &\iff a : \mathsf{U} \qquad \text{(every type has } \bot\text{)} \\ \mathsf{U} : \mathsf{U} &= \top \\ \mathsf{Fun}(g) : \mathsf{Pi}(a, f) &\iff \mathsf{FinMemFun}(g, a, f) \ \wedge \ \mathsf{Coh}(\mathsf{Fun}(g)) \ \wedge \ \mathsf{Pi}(a, f) : \mathsf{U} \\ \mathsf{Pi}(a, f) : \mathsf{U} &\iff a : \mathsf{U} \ \wedge \ \mathsf{FinMemAllU}(f, a) \ \wedge \ \mathsf{CohFunTail}(f) \end{aligned}$$

where $\mathsf{FinMemFun}(g, a, f)$ checks that every edge $(u_i, v_i) \in g$ satisfies $u_i : a$ and $v_i : \mathsf{ev}(f, u_i)$, and $\mathsf{FinMemAllU}(f, a)$ checks that every edge $(a_i, b_i) \in f$ satisfies $a_i : a$ and $b_i : \mathsf{U}$.

Key structural properties:

- $u : a$ implies $\mathsf{Coh}(u)$, $\mathsf{Coh}(a)$, and $a : \mathsf{U}$.
- Membership is monotone: if $u : a$ and $a \leq a'$ with $a' : \mathsf{U}$, then $u : a'$.

7 Selections

A *selection* of a finite function f is a compatible subset of edges from f, yielding an aggregate key u and aggregate value v. Formally, $\mathsf{Selection}(f, u, v)$ is an inductive type with three constructors:

$$\frac{}{\mathsf{Selection}(\mathsf{nil}, \bot, \bot)}\ \textsc{Sel-Nil} \qquad \frac{\mathsf{Selection}(g, u, v)}{\mathsf{Selection}((a_i, b_i)::g,\ u,\ v)}\ \textsc{Sel-Skip}$$

$$\frac{\mathsf{Comp}(a_i, u) \qquad \mathsf{Comp}(b_i, v) \qquad \mathsf{Selection}(g, u, v)}{\mathsf{Selection}((a_i, b_i)::g,\ a_i \sqcup u,\ b_i \sqcup v)}\ \textsc{Sel-Take}$$

The compatibility conditions in Sel-Take ensure that the accumulated u and v remain coherent. A key property is that $\mathsf{Selection}(f, u, v)$ implies $v \leq \mathsf{ev}(f, u)$.

Why selections rather than individual edges? The evaluation $\mathsf{ev}(f, u)$ is defined as a supremum: it aggregates the outputs of *all* edges in f whose keys lie below u. Working with single edges would not capture this aggregate character. A selection encodes a finite compatible subfamily of edges whose joins approximate $\mathsf{ev}(f, u)$ from below, and the compatibility check guarantees that these joins are well-defined (i.e. coherent). This is the mechanism that lets the relational semantics and the validity relation handle Π-types and λ-terms in a purely finite way, without ever computing an infinite supremum.

8 Relational semantics on finite approximants

Definition 2 (Evaluation relation)**.** *Given a raw term M in context of size n, a finite environment ρ (mapping each variable to a FinEl), and a finite element u, the relation $[\![M]\!]_\rho \ni u$ ("M evaluates to at least u in ρ") is defined by recursion on M:*

- ***Variable:*** $[\![x_i]\!]_\rho \ni u$ *iff* $\mathsf{Coh}(u)$ *and* $u \leq \rho(i)$.
- ***Universe:*** $[\![\mathsf{U}]\!]_\rho \ni u$ *iff* $\mathsf{Coh}(u)$ *and* $u \leq \mathsf{U}$.

- ***Application:*** $[\![M\ N]\!]_\rho \ni \bot$ *always holds. For non-*$\bot$ *output* u*: there exists* v *such that* $[\![N]\!]_\rho \ni v$ *and* $[\![M]\!]_\rho \ni \mathsf{Fun}([(v,u)])$*. (One-edge form: the function* M *maps input* v *to output* u*.)*

- ***Lambda:*** $[\![\lambda(x{:}A)M]\!]_\rho \ni \bot$ *always holds.* $[\![\lambda(x{:}A)M]\!]_\rho \ni \mathsf{Fun}(g)$ *requires a domain code* a *with* $\mathsf{Coh}(\mathsf{Fun}(g))$*,* $a : \mathsf{U}$*,* $[\![A]\!]_\rho \ni a$*, and for every selection of* g *with key* u *and value* v*, there exists* $x \le u$ *with* $x : a$ *and* $[\![M]\!]_{\rho[x]} \ni v$*.*

- ***Pi:*** $[\![\Pi(x{:}A)B]\!]_\rho \ni \bot$ *always holds.* $[\![\Pi(x{:}A)B]\!]_\rho \ni \mathsf{Pi}(a,f)$ *requires* $\mathsf{Coh}(\mathsf{Pi}(a,f))$*,* $[\![A]\!]_\rho \ni a$*, and a witness* a' *with* $[\![A]\!]_\rho \ni a'$ *such that for every selection of* f *with key* u *and value* v*, there exists* $x \le u$ *with* $x : a'$ *and* $[\![B]\!]_{\rho[x]} \ni v$*.*

The relation is purely finite and has no reference to ideal semantic functions. Key properties:

- *Coherence extraction*: $[\![M]\!]_\rho \ni u$ implies $\mathsf{Coh}(u)$.
- *Compatibility*: if $[\![M]\!]_\rho \ni u$ and $[\![M]\!]_\rho \ni v$ with $\mathsf{Coh}(\rho)$, then $\mathsf{Comp}(u,v)$.
- *Monotonicity in* ρ: if $\rho \le \rho'$ pointwise and $[\![M]\!]_\rho \ni u$, then $[\![M]\!]_{\rho'} \ni u$.
- *Downward closure*: $[\![M]\!]_\rho \ni u$ and $u' \le u$ with $\mathsf{Coh}(u')$ implies $[\![M]\!]_\rho \ni u'$.
- *Sup closure*: $[\![M]\!]_\rho \ni u$ and $[\![M]\!]_\rho \ni v$ implies $[\![M]\!]_\rho \ni u \sqcup v$ (when the environment is coherent).

9 Soundness of the relational semantics

Definition 3 (Fitting environment)**.** *An environment* ρ fits *a context* Γ *(written* $\mathsf{Fits}(\Gamma,\rho)$*) when, for each variable* $x_i : A_i$ *in* Γ*, there exists a code* a' *such that* $\rho(i) : a'$ *and* $[\![A_i]\!]_{\rho\restriction i} \ni a'$*.*

Definition 4 (Typed evaluation)**.** $\mathsf{Typed}(M,A,\rho,u)$ *asserts that* u *can be enlarged to some* $u' \ge u$ *which is a member of some code* a' *that* A *evaluates to: there exist* u' *and* a' *such that* $u \le u'$*,* $[\![M]\!]_\rho \ni u'$*,* $u' : a'$*, and* $[\![A]\!]_\rho \ni a'$*.*

Theorem 5 (Typing soundness — Theorem 1 of [1])**.** *If* $\Gamma \vdash M : A$ *and* $\mathsf{Fits}(\Gamma,\rho)$*, then for every* u *with* $[\![M]\!]_\rho \ni u$*, we have* $\mathsf{Typed}(M,A,\rho,u)$*.*

Theorem 6 (Conversion soundness)**.** *If* $\Gamma \vdash M = N : A$ *and* $\mathsf{Fits}(\Gamma,\rho)$*, then:*

1. *Typing:* $\mathsf{Typed}(M,A,\rho,u)$ *for all* $[\![M]\!]_\rho \ni u$*, and likewise for* N*.*
2. *Bidirectional transfer: for all* u*,* $[\![M]\!]_\rho \ni u \iff [\![N]\!]_\rho \ni u$*.*

The proofs proceed by mutual induction on the typing and conversion derivations. The β and function extensionality cases use the evaluation–substitution lemma: if $[\![M]\!]_{\rho[v]} \ni u$ and $[\![N]\!]_\rho \ni v$, then $[\![M[N/x]]\!]_\rho \ni u$ (and conversely). Formalised in `TypingSemantics.agda` and `LemmaForTS.agda`, with 0 postulates.

10 Logical relation (validity)

The validity relation is defined by mutual structural recursion on the type code $a \in \mathsf{FinEl}$. It connects the syntactic judgments to the finite semantics.

Definition 7 (Validity)**.** *For a context Γ, terms M, A, a realizer u, and a type code a, we define mutually:*

Term validity $\mathsf{Val}(\Gamma, M, A, u, a)$ *by case split on a:*

- *At $a = \bot$: always $\top$ (trivially valid).*
- *At $a = \mathsf{U}$: reduces to type validity $\mathsf{ValTy}(\Gamma, M, u)$.*
- *At $a = \mathsf{Pi}(b, f)$ with $u = \mathsf{Fun}(g)$: a Σ-type containing*
  1. *type validity of A at $\mathsf{Pi}(b, f)$ (head-reduces to some $\Pi\, A_0\, B_0$),*
  2. *$\mathsf{Coh}(\mathsf{Fun}(g))$ and $\mathsf{FinMemFun}(g, b, f)$,*
  3. *for every selection of g with key u' and value v, and argument N with $\mathsf{Val}(\Gamma, N, A_0, u', b)$:*

$$\mathsf{Val}(\Gamma, M\ N, B_0[N], v, \mathsf{ev}(f, u')).$$

- *At $a = \mathsf{Pi}(b, f)$ with $u \neq \mathsf{Fun}(g)$: always $\top$.*

Type validity $\mathsf{ValTy}(\Gamma, M, u)$ *at $u = \mathsf{Pi}(b, f)$: a Σ-type containing terms A, B such that $M \rightsquigarrow^* \Pi(x{:}A)B$ in Γ, $\mathsf{CohFunTail}(f)$, $\mathsf{FinMemAllU}(f, b)$, type validity $\mathsf{ValTy}(\Gamma, A, b)$, and for every selection of f with key u' and value v, and argument N with $\mathsf{Val}(\Gamma, N, A, u', b)$: $\mathsf{ValTy}(\Gamma, B[N], v)$.*

Equality validity $\mathsf{EqVal}(\Gamma, M, N, A, u, a)$ *bundles validity of both M and N, plus type-equality data at Π-codes.*

The key feature of this logical relation is that it recurses on the type code a rather than on syntactic types. Since $\mathsf{ev}(f, u')$ is structurally smaller than $\mathsf{Pi}(b, f)$ (its rank is bounded by $\mathsf{rk}_{\mathsf{fun}}(f) < \mathsf{rk}(\mathsf{Pi}(b, f))$), the recursion is well-founded despite the type-in-type universe.

Monotonicity properties. Three mutual monotonicity families are proved:

- *Downward in a*: if $a_0 \leq a_1$, $u : a_0$, $\mathsf{Coh}(a_0)$, and $a_1 : \mathsf{U}$, then $\mathsf{Val}(\Gamma, M, A, u, a_1) \Rightarrow \mathsf{Val}(\Gamma, M, A, u, a_0)$.
- *Upward in a*: if $a_0 \leq a_1$, $u : a_0$, $u : a_1$, $\mathsf{Coh}(a_0)$, $\mathsf{Coh}(a_1)$, and $\mathsf{ValTy}(\Gamma, A, a_1)$, then $\mathsf{Val}(\Gamma, M, A, u, a_0) \Rightarrow \mathsf{Val}(\Gamma, M, A, u, a_1)$.
- *Restriction in u*: if $u' \leq u$, $u' : a$, and $u : a$, then $\mathsf{Val}(\Gamma, M, A, u, a) \Rightarrow \mathsf{Val}(\Gamma, M, A, u', a)$.

Formalised in `Validity.agda` with 0 postulates.

11 Adequacy

The adequacy theorem is the bridge between the syntactic typing judgments and the semantic validity relation. It is proved using the two-substitution technique of [1], Theorem 2.

Definition 8 (Bundled validity)**.** $\mathsf{Val2}(\Gamma, M, A, u, a)$ *refines* Val *by requiring $\Gamma \vdash M : A$ at the leaves (at $\bot$-realizer or $\bot$-code positions). Similarly,* $\mathsf{EqVal2}$ *requires $\Gamma \vdash M = N : A$. These bundled versions are defined in* ***`Validity2.agda`****.*

Theorem 9 (Adequacy — two-substitution form)**.** *The following are proved by mutual induction on typing/conversion derivations, instantiated at two substitutions σ_1, σ_2 simultaneously:*

1. *If $\Gamma \vdash M : A$, then for all codes u, a with $[\![M]\!]_\rho \ni u$ and $[\![A]\!]_\rho \ni a$:*

$$\mathsf{Val2}(\Delta,\, M[\sigma],\, A[\sigma],\, u,\, a).$$

2. *If $\Gamma \vdash M = N : A$, then under the same conditions:*

$$\mathsf{EqVal2}(\Delta,\, M[\sigma],\, N[\sigma],\, A[\sigma],\, u,\, a).$$

The proof uses the substitution lemma (well-typed substitutions preserve typing and conversion), context conversion, the soundness theorems 5 and 6, and the monotonicity properties of the validity relation. Key cases:

- Beta: uses the evaluation–substitution correspondence and β-expansion/contraction lemmas for Val2.
- Funext: uses weakening for EvalRel and the selection-based Pi clauses.
- Conv: uses the bidirectional transfer from conversion soundness and the downward/upward monotonicity of Val2.
- Pi: constructs a $\mathsf{Pi}(b, f)$-realizer by building selections from the domain and codomain adequacy hypotheses.

Formalised in `Adequacy2.agda` with 0 postulates.

12 Main result: Pi injectivity

Theorem 10 (Corollary 6; Coquand–Huber 2018, p. 661)**.** *If $\Gamma \vdash A_0 = \Pi(x{:}B_1)F_1 : \mathsf{U}$, then there exist B_0, F_0 such that*

1. *$A_0 \rightsquigarrow^* \Pi(x{:}B_0)F_0$,*
2. *$\Gamma \vdash B_0 = B_1 : \mathsf{U}$,*
3. *$\Gamma, x{:}B_0 \vdash F_0 = F_1 : \mathsf{U}$.*

Corollary 11 (Pi–Pi injectivity)**.** *If $\Gamma \vdash \Pi(x{:}A_0)B_0 = \Pi(x{:}A_1)B_1 : \mathsf{U}$, then*

$$\Gamma \vdash A_0 = A_1 : \mathsf{U} \qquad \textit{and} \qquad \Gamma, x{:}A_0 \vdash B_0 = B_1 : \mathsf{U}.$$

Proof outline (Theorem). The proof proceeds by domain-theoretic adequacy.

1. From $\Gamma \vdash A_0 = \Pi B_1\, F_1 : \mathsf{U}$, extract $\Gamma \vdash A_0 : \mathsf{U}$ by presupposition (SubstitutionLemma).
2. Apply the soundness theorem (convSound') at the *bottom environment* $\rho_\perp$ (mapping every variable to $\perp$) to obtain a bidirectional evaluation transfer: for every finite approximation u,

$$\mathsf{EvalRel}(\Pi B_1\, F_1, \rho_\perp, u) \implies \mathsf{EvalRel}(A_0, \rho_\perp, u).$$

3. Instantiate with $u = \mathsf{PiCode}(\bot, \mathsf{nil})$, which is coherent because $\mathsf{CoherentFunTail}(\mathsf{nil}) = \top$. The evaluation $\mathsf{EvalRel}(\Pi B_1\, F_1, \rho_\bot, \mathsf{PiCode}(\bot, \mathsf{nil}))$ is constructible trivially (domain $\bot$, empty function graph, vacuous selection body). Transfer gives $\mathsf{EvalRel}(A_0, \rho_\bot, \mathsf{PiCode}(\bot, \mathsf{nil}))$.

4. Apply the bundled equality adequacy theorem (adequacyEqSub2) with the identity substitution at $\rho_\bot$ to obtain

$$\mathsf{EqVal2}\ \Gamma\ A_0\ (\Pi B_1\, F_1)\ \mathsf{U}\ (\mathsf{PiCode}(\bot, \mathsf{nil}))\ \mathsf{UCode}.$$

5. This unfolds to EqValTyPi2, a Σ-type containing:

   - $\mathsf{Red}\ \Gamma\ A_0\ (\Pi\, B_0\, F_0)\ \mathsf{U}$ (giving part 1),
   - $\Gamma \vdash B_0 = B_1' : \mathsf{U}$ and $\Gamma, x{:}B_0 \vdash F_0 = F_1' : \mathsf{U}$,
   - $\mathsf{Red}\ \Gamma\ (\Pi B_1\, F_1)\ (\Pi B_1'\, F_1')\ \mathsf{U}$.

   Since Π is a head normal form, Red-unique-Pi forces $B_1' = B_1$ and $F_1' = F_1$, giving parts 2 and 3.

□

Proof of Corollary. Apply the theorem to $\Gamma \vdash \Pi A_0\, B_0 = \Pi A_1\, B_1 : \mathsf{U}$. This yields B_0', F_0' with $\Pi A_0\, B_0 \rightsquigarrow^* \Pi B_0'\, F_0'$. Since Π is a head normal form, Red-unique-Pi gives $B_0' = A_0$ and $F_0' = B_0$, so the domain and codomain conversions become $\Gamma \vdash A_0 = A_1 : \mathsf{U}$ and $\Gamma, x{:}A_0 \vdash B_0 = B_1 : \mathsf{U}$. □

Key intermediate lemmas. The proof above relies on several results that deserve explicit mention:

- *Presupposition* (`SubstitutionLemma.agda`): from a conversion judgment $\Gamma \vdash M = N : A$ one can extract $\Gamma \vdash M : A$, $\Gamma \vdash N : A$, and $\Gamma \vdash A : \mathsf{U}$. This is proved by mutual induction on typing and conversion, using the substitution and renaming lemmas.
- *Conversion soundness* (`convSound'` in `TypingSemantics.agda`): if $\Gamma \vdash M = N : A$ and $\mathsf{EvalRel}(N, \rho, u)$, then $\mathsf{EvalRel}(M, \rho, u)$ (and vice versa).
- *Bundled adequacy* (`adequacyEqSub2` in `Adequacy2.agda`): the bridge from EvalRel to the validity relation EqVal2, which packages both a semantic value and the corresponding syntactic typing/conversion derivation.
- *Reduction uniqueness for* Π (`Red-unique-Pi`): since Π is a head normal form, $\Pi\, A\, B \rightsquigarrow^* \Pi\, A'\, B'$ implies $A = A'$ and $B = B'$. This is the step that turns the "up to reduction" output of adequacy into exact equalities on the domain and codomain.

The entire proof is formalised in Agda with `--without-K --exact-split`, no `--type-in-type`, and no postulates (in the files on which `PiInjectivity.agda` depends).

References

[1] T. Coquand and S. Huber, "An adequacy theorem for dependent type theory," *Archive for Mathematical Logic*, vol. 57, pp. 649–676, 2018.